

- Aqui está um exemplo simples de um **verificador de saldo em Python**, simulando uma conta bancária:

```
class ContaBancaria:
```

```
    def __init__(self, titular, saldo_inicial=0):
```

```
        self.titular = titular
```

```
        self.saldo = saldo_inicial
```

```
    def verificar_saldo(self):
```

```
        print(f"Saldo atual de {self.titular}: R$ {self.saldo:.2f}")
```

```
    def depositar(self, valor):
```

```
        if valor > 0:
```

```
            self.saldo += valor
```

```
            print(f"Depósito de R$ {valor:.2f} realizado com sucesso.")
```

```
        else:
```

```
            print("Valor inválido para depósito.")
```

```
    def sacar(self, valor):
```

```
        if valor <= 0:
```

```
            print("Valor inválido para saque.")
```

```
        elif valor > self.saldo:
```

```
            print("Saldo insuficiente.")
```

```
        else:
```

```
            self.saldo -= valor
```

```
            print(f"Saque de R$ {valor:.2f} realizado com sucesso.")
```

Exemplo de uso

```
conta = ContaBancaria("Rafael", 1000)
```

```
conta.verificar_saldo()
```

```
conta.depositar(500)
```

```
conta.verificar_saldo()
```

```
conta.sacar(300)
```

```
conta.verificar_saldo()
```

O que esse código faz:

- * Cria uma conta com nome e saldo inicial

- * Permite **verificar saldo**

- * Permite **depositar dinheiro**

- * Permite **sacar dinheiro com validação**

▪ Claro! Vou refazer o código como um estudante iniciante faria, explicando **linha por linha** com comentários simples:

Criando uma classe chamada ContaBancaria

```
class ContaBancaria:
```

```
    # Método construtor: executado quando criamos uma conta
```

```
    def __init__(self, titular, saldo_inicial=0):
```

```
        self.titular = titular    # Guarda o nome do dono da conta
```

```
        self.saldo = saldo_inicial    # Guarda o saldo inicial (começa com 0 se não  
informar)
```

```
# Método para verificar o saldo
```

```
def verificar_saldo(self):
```

```
    # Mostra o saldo atual formatado com duas casas decimais
```

```
    print(f"Saldo atual de {self.titular}: R$ {self.saldo:.2f}")
```

```
# Método para depositar dinheiro
```

```
def depositar(self, valor):
```

```
    if valor > 0:          # Verifica se o valor é positivo
```

```
        self.saldo += valor    # Soma o valor ao saldo atual
```

```
        print(f"Depósito de R$ {valor:.2f} realizado com sucesso.")
```

```
    else:
```

```
        print("Valor inválido para depósito.") # Caso o valor seja negativo ou zero
```

```
# Método para sacar dinheiro
```

```
def sacar(self, valor):
```

```
    if valor <= 0:          # Verifica se o valor é válido
```

```
        print("Valor inválido para saque.")
```

```
    elif valor > self.saldo:    # Verifica se há saldo suficiente
```

```
        print("Saldo insuficiente.")
```

```
    else:
```

```
        self.saldo -= valor    # Subtrai o valor do saldo
```

```
        print(f"Saque de R$ {valor:.2f} realizado com sucesso.")
```

```
# Criando uma conta para testar o programa
```

```
conta = ContaBancaria("Rafael", 1000) # Nome do titular e saldo inicial
```

Chamando os métodos para ver como funciona

conta.verificar_saldo() # Mostra o saldo inicial

conta.depositar(500) # Deposita 500 reais

conta.verificar_saldo() # Mostra o novo saldo

conta.sacar(300) # Saca 300 reais

conta.verificar_saldo() # Mostra o saldo final

Resumindo de forma simples:

* **Classe** = modelo da conta bancária

* **self** = representa a própria conta

* **saldo** = dinheiro disponível

* **métodos** = ações (ver saldo, depositar, sacar)